Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Модели решения задач в И С»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:
Студент 3 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Ход работы

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

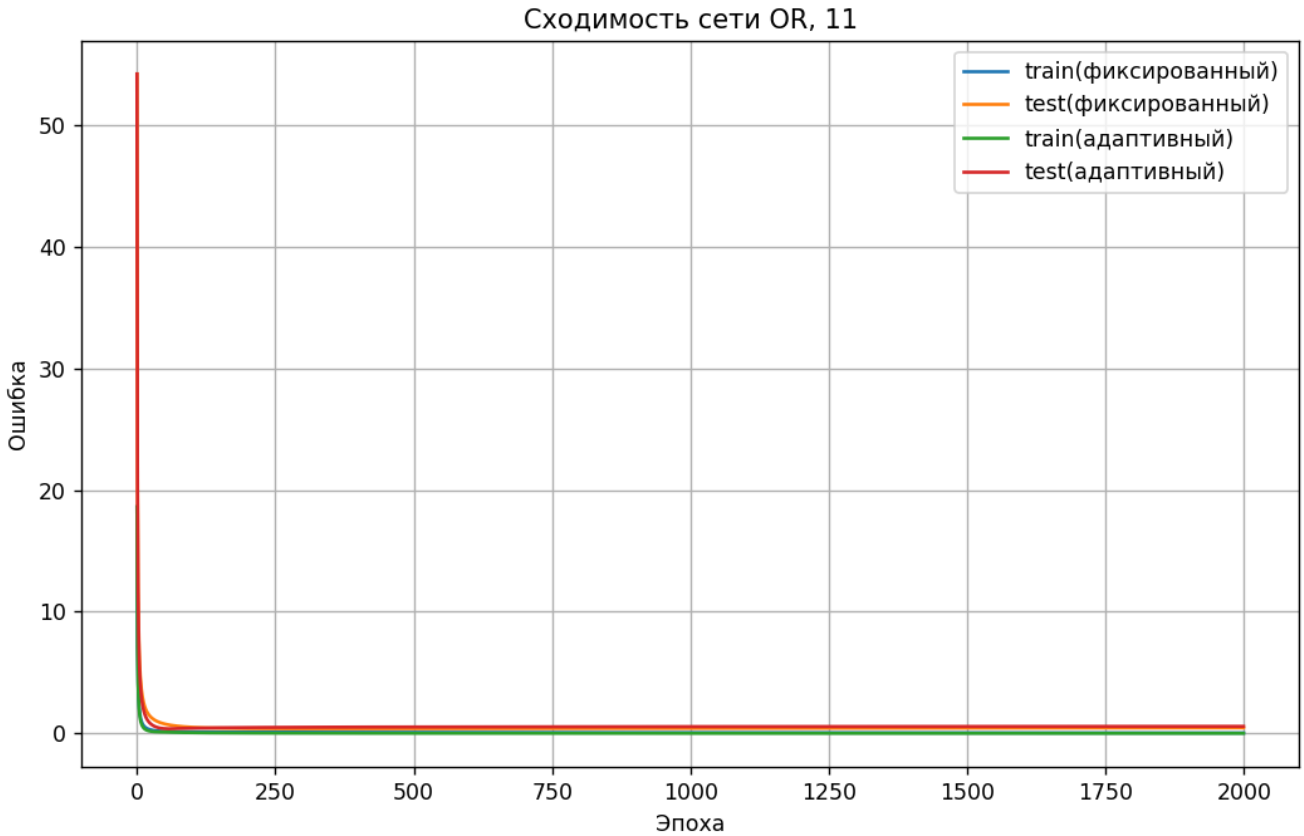
Вариант:

№ of variant	n	Task
25	11	OR

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

N = 11
np.random.seed(42)

X_all = np.array(list(itertools.product([0, 1], repeat=N)), dtype=float)
y_all = np.any(X_all == 1, axis=1).astype(float)

print(f"Всего примеров: {len(X_all)}")
print(f"Класс 1: {int(y_all.sum())} | Класс 0: {int((1-y_all).sum())}\n")

idx_zero = np.where(y_all == 0)[0]
idx_one = np.where(y_all == 1)[0]


train_idx = np.concatenate([
    idx_zero,
    np.random.choice(idx_one, size=499, replace=False)
```

```
])

test_idx = np.array([i for i in range(len(X_all)) if i not in train_idx])

X_train, y_train = X_all[train_idx], y_all[train_idx]
X_test, y_test = X_all[test_idx], y_all[test_idx]

print(f"Обучающая выборка: {len(X_train)} (класс 1: {int(y_train.sum())})")
print(f"Тестовая выборка: {len(X_test)} (класс 1: {int(y_test.sum())})\n")

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -50, 50)))

def total_error(X, y, w, b):
    yp = sigmoid(X @ w + b)
    return 0.5 * np.sum((y - yp) ** 2)

def accuracy(X, y, w, b):
    yp = sigmoid(X @ w + b)
    return np.mean((yp >= 0.5).astype(float) == y)

def train(X_tr, y_tr, X_te, y_te, mode='фиксированный', alpha=0.05, max_epochs=2000,
tol=1e-4):
    rng = np.random.default_rng(1)
    w = rng.uniform(0.0, 0.5, size=N)
    b = -5.0

    train_err, test_err = [], []
    lr = alpha
    prev_err = float('inf')

    for epoch in range(max_epochs):
        for i in rng.permutation(len(X_tr)):
            xi, yi = X_tr[i], y_tr[i]
            out = sigmoid(xi @ w + b)
            delta = (yi - out) * out * (1 - out)
            w += lr * delta * xi
            b += lr * delta

        err_tr = total_error(X_tr, y_tr, w, b)
        err_te = total_error(X_te, y_te, w, b)
        train_err.append(err_tr)
        test_err.append(err_te)

        if mode == 'адаптивный':
            if err_tr < prev_err:
                lr = min(lr * 1.05, 1.0)
            else:
                lr = max(lr * 0.5, 1e-5)
        prev_err = err_tr

        if err_tr < tol:
            print(f"{mode.capitalize()} обучение завершено на эпохе {epoch+1}")
            break

    return w, b, train_err, test_err, epoch+1

print("\nОбучение: фиксированный шаг")
w_fix, b_fix, tr_fix, te_fix, ep_fix = train(
    X_train, y_train, X_test, y_test, mode='фиксированный', alpha=0.05
)

print("\nОбучение: адаптивный шаг")
w_adapt, b_adapt, tr_adapt, te_adapt, ep_adapt = train(
```

```
    X_train, y_train, X_test, y_test, mode='адаптивный', alpha=0.05
)

print(f"\nКоличество эпох:")
print(f"  (фиксированный) {ep_fix}")
print(f"  (адаптивный) {ep_adapt}\n")

print(f"Точность на обучении:")
print(f"  (фиксированный) {accuracy(X_train, y_train, w_fix, b_fix)*100:.2f}%")
print(f"  (адаптивный) {accuracy(X_train, y_train, w_adapt, b_adapt)*100:.2f}%\n")

print(f"Точность на тесте:")
print(f"  (фиксированный) {accuracy(X_test, y_test, w_fix, b_fix)*100:.2f}%")
print(f"  (адаптивный) {accuracy(X_test, y_test, w_adapt, b_adapt)*100:.2f}%\n")

print("Веса (фиксированный шаг):")
for i, wi in enumerate(w_fix):
    print(f"w{i+1} = {wi:.4f}")
print(f"b = {b_fix:.4f}\n")

print("Веса (адаптивный шаг):")
for i, wi in enumerate(w_adapt):
    print(f"w{i+1} = {wi:.4f}")
print(f"b = {b_adapt:.4f}\n")

plt.figure(figsize=(10, 6))

plt.plot(tr_fix, label="train(фиксированный)")
plt.plot(te_fix, label="test(фиксированный)")
plt.plot(tr_adapt, label="train(адаптивный)")
plt.plot(te_adapt, label="test(адаптивный)")

plt.xlabel("Эпоха")
plt.ylabel("Ошибка")
plt.title("Сходимость сети OR, 11")
plt.legend()
plt.grid(True)
plt.show()

def predict(vec, w, b):
    x = np.array(vec, dtype=float)
    prob = float(sigmoid(x @ w + b))
    return prob, int(prob >= 0.5)

print("\nПроверка")
print(f"Введите {N} чисел (0 или 1) через пробел, либо 'q' для выхода")

while True:
    s = input("Вход: ")
    if s.lower() == 'q':
        break

    parts = s.split()
    if len(parts) != N:
        print("Ошибка: требуется 11 значений\n")
        continue

    vec = list(map(int, parts))
    prob, cls = predict(vec, w_fix, b_fix)
    true_val = int(np.any(vec))

    print(f"Вероятность принадлежности к классу 1: {prob:.4f}")
    print(f"Предсказанный класс: {cls} | Истинный класс: {true_val}\n")
```



На графике видно, что при фиксированном шаге обучение идет медленнее, но ошибка постепенно уменьшается, а при адаптивном шаге сеть обучается быстрее и быстрее сходится к хорошему результату. Ошибки на обучающей и тестовой выборках почти одинаковые, значит модель нормально обобщает данные. Несмотря на сильный дисбаланс (всего один нулевой пример), сигмоидальный персептрон успешно обучился операции OR на 11 входах, что доказывает способность восстанавливать полную таблицу истинности по частичной выборке.

```
Всего примеров: 2048
Класс 1: 2047 | Класс 0: 1

Обучающая выборка: 500 (класс 1: 499)
Тестовая выборка: 1548 (класс 1: 1548)


Обучение: фиксированный шаг

Обучение: адаптивный шаг

Количество эпох:
  (фиксированный) 2000
  (адаптивный) 2000

Точность на обучении:
  (фиксированный) 100.00%
  (адаптивный) 100.00%

Точность на тесте:
  (фиксированный) 100.00%
  (адаптивный) 99.94%
```

```
Веса (фиксированный шаг):
w1 = 2.1858
w2 = 2.2549
w3 = 2.5073
w4 = 2.3455
w5 = 3.5635
w6 = 2.2487
w7 = 2.0694
w8 = 2.4262
w9 = 2.1262
w10 = 1.8960
w11 = 2.4257
b = -1.3353

Веса (адаптивный шаг):
w1 = 3.7678
w2 = 3.6839
w3 = 4.2320
w4 = 3.7808
w5 = 6.7212
w6 = 3.7085
w7 = 3.3223
w8 = 4.1514
w9 = 3.5269
w10 = 2.9936
w11 = 4.2297
b = -2.9990
```

```
Проверка
Введите 11 чисел (0 или 1) через пробел, либо 'q' для выхода
Вход: 0 0 0 0 0 0 0 0 0 0 0 0
Ошибка: требуется 11 значений

Вход: 0 0 0 0 0 0 0 0 0 0 0
Вероятность принадлежности к классу 1: 0.2083
Предсказанный класс: 0 | Истинный класс: 0

Вход: 1 0 0 0 1 1 1 1 0 0 1
Вероятность принадлежности к классу 1: 1.0000
Предсказанный класс: 1 | Истинный класс: 1
```

Вывод: Реализовала нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнила эффективность обучения при различных начальных условиях и параметрах шага.